

ui.grid 全面详细解析

`ui.grid` 是 NiceGUI 框架中的网格布局容器组件，核心作用是按网格形式排列子元素，支持灵活的行列配置、单元格跨列、样式自定义等功能，适配从简单数据展示到复杂页面布局的多种场景。以下从核心特性、使用方法、高级功能、属性与方法等方面进行全面解析。

一、核心概念与初始化参数

1. 核心定位

`ui.grid` 基于 CSS Grid 布局实现，通过定义行列规则，自动将子元素填充到网格单元格中，相比传统布局更简洁、灵活，无需手动计算元素位置和间距。

2. 初始化关键参数

初始化 `ui.grid` 时，最核心的两个参数用于定义网格的行列结构，支持两种配置方式（直接指定数量或使用 CSS 语法）：

参数	说明	示例
<code>rows</code>	网格行数，或 CSS <code>grid-template-rows</code> 属性值	- 数量: <code>rows=3</code> (3 行等分布局) - CSS 语法: <code>rows='auto 1fr 80px'</code> (第一行自适应内容、第二行占满剩余空间、第三行固定 80px 高)
<code>columns</code>	网格列数，或 CSS <code>grid-template-columns</code> 属性值	- 数量: <code>columns=2</code> (2 列等分布局) - CSS 语法: <code>columns='auto 1fr 2fr'</code> (第一列自适应、第二列占 1 份剩余空间、第三列占 2 份剩余空间)

基础使用示例 (2 列布局展示用户信息)：

```
from nicegui import ui

with ui.grid(columns=2): # 2 列等分布局
    ui.label('Name:')
    ui.label('Tom')
    ui.label('Age:')
    ui.label('42')
    ui.label('Height:')
    ui.label('1.80m')

ui.run()
```

二、行列布局的灵活配置（CSS 语法详解）

当 `rows` 或 `columns` 使用字符串格式时，支持 CSS Grid 所有合法维度值，核心常用语法如下：

维度值	作用	示例效果
auto	元素尺寸自适应内容（内容越长，行列越宽 / 高）	<code>columns='auto 1fr'</code> ：第一列宽度适配文本长度，第二列占满剩余空间
fr	剩余空间分配单位（支持 1fr、2fr 等比例）	<code>columns='1fr 2fr'</code> ：两列总宽度为父容器宽度，比例 1:2 分配空间
固定尺寸	直接指定像素（px）、百分比（%）等	<code>columns='80px 20% 1fr'</code> ：第一列固定 80px、第二列占父容器 20%、第三列占剩余空间

自定义列布局示例：

```
from nicegui import ui

# 4 列布局: auto + 80px + 1fr + 2fr, 占满父容器宽度, 无间距
with ui.grid(columns='auto 80px 1fr 2fr').classes('w-full gap-0'):
    for _ in range(3): # 重复 3 行展示列类型
        ui.label('auto').classes('border p-1') # 自适应列
        ui.label('80px').classes('border p-1') # 固定 80px 列
        ui.label('1fr').classes('border p-1') # 1 份剩余空间列
        ui.label('2fr').classes('border p-1') # 2 份剩余空间列

ui.run()
```

三、高级功能：单元格跨列

`ui.grid` 支持通过 Tailwind 类或直接 CSS 样式，让单元格跨越多列（类似表格的 `colspan`），满足复杂布局需求：

1. 常用跨列语法

实现方式	语法	说明
Tailwind 类	<code>col-span-N</code> （N 为跨列数）	适用于跨列数 ≤ 12 的场景（Tailwind 内置类），如 <code>col-span-8</code> （跨 8 列）
自定义跨列	<code>col-[span_N]</code> （N 为任意整数）	适用于跨列数 >12 的场景，如 <code>col-[span_15]</code> （跨 15 列）
直接 CSS	<code>.style('grid-column: span N / span N')</code>	通用语法，N 为跨列数，兼容所有场景

2. 跨列示例（16 列网格布局）

```
from nicegui import ui

# 16 列网格，占满父容器，无间距
with ui.grid(columns=16).classes('w-full gap-0'):
    ui.label('full').classes('col-span-full border p-1') # 跨所有列（16 列）
    ui.label('8').classes('col-span-8 border p-1') # 跨 8 列
    ui.label('8').classes('col-span-8 border p-1') # 跨 8 列
    ui.label('12').classes('col-span-12 border p-1') # 跨 12 列
    ui.label('4').classes('col-span-4 border p-1') # 跨 4 列
    ui.label('15').classes('col-[span_15] border p-1') # 跨 15 列（自定义跨列数）
    ui.label('1').classes('col-span-1 border p-1') # 跨 1 列

ui.run()
```

四、核心属性

`ui.grid` 继承自 NiceGUI 基础 `Element` 类，拥有以下常用属性（部分为 2.16.0+ 版本新增）：

属性名	类型	说明	
<code>classes</code>	<code>Classes[Self]</code>	元素的 Tailwind/Quasar 类（用于设置宽度、间距、边框等样式）	
<code>client</code>	<code>Client</code>	该元素所属的客户端实例	
<code>html_id</code>	<code>str</code>	HTML DOM 中的元素 ID（2.16.0+ 新增）	
<code>is_deleted</code>	<code>bool</code>	元素是否已被删除	
<code>is_ignoring_events</code>	<code>bool</code>	元素是否正在忽略事件	
<code>parent_slot</code>	<code>`Slot`</code>	<code>None`</code>	元素的父插槽（可修改）
<code>props</code>	<code>Props[Self]</code>	元素的 Quasar props（HTML 属性）	
<code>style</code>	<code>Style[Self]</code>	元素的自定义 CSS 样式	
<code>visible</code>	<code>BindableProperty</code>	元素可见性（支持双向绑定）	

属性使用示例（设置网格宽度、间距和边框）：

```
with ui.grid(columns=2).classes('w-1/2 mx-auto gap-4 border p-4'):
    # w-1/2: 宽度为父容器的 50%; mx-auto: 水平居中; gap-4: 单元格间距 4px; border p-4: 边框和内边距
    ui.label('Name:')
    ui.label('Tom')
```

五、常用方法

`ui.grid` 提供丰富的方法用于动态操作元素、绑定事件、管理样式等，以下是高频使用方法：

1. 样式与布局相关

方法	作用	示例
<code>classes()</code>	为网格添加 / 修改 Tailwind 类	<code>.classes('w-full gap-2 border')</code>
<code>style()</code>	为网格添加自定义 CSS 样式	<code>.style('grid-template-rows: auto 1fr; gap: 8px')</code>
<code>default_classes()</code>	为所有该类型网格设置默认类（需在实例化前调用）	<code>ui.grid.default_classes(add='gap-3')</code> （所有 grid 默认间距 3px）
<code>default_style()</code>	为所有该类型网格设置默认 CSS 样式	<code>ui.grid.default_style(add='background: #f5f5f5')</code>

2. 元素管理相关

方法	作用	示例
<code>clear()</code>	删除所有子元素	<code>grid.clear()</code> （清空网格内所有标签 / 组件）
<code>remove(element)</code>	删除指定子元素（支持元素实例或 ID）	<code>grid.remove(label_instance)</code>
<code>delete()</code>	删除网格本身及所有子元素	<code>grid.delete()</code>
<code>move(target_container)</code>	将网格移动到另一个容器中	<code>grid.move(ui.card())</code> （将网格移入卡片组件）

3. 事件与绑定相关

方法	作用	示例
<code>on(type, handler)</code>	绑定事件（如点击、鼠标悬浮等）	<code>grid.on('click', lambda: print('Grid clicked'))</code>
<code>bind_visibility(target_object, target_name)</code>	双向绑定可见性到目标对象属性	<code>grid.bind_visibility(data, 'show_grid')</code> （ <code>data.show_grid</code> 控制网格显示 / 隐藏）

方法	作用	示例
<code>bind_visibility_from()</code>	单向绑定可见性（从目标对象到网格）	<code>grid.bind_visibility_from(data, 'show_grid')</code>
<code>bind_visibility_to()</code>	单向绑定可见性（从网格到目标对象）	<code>grid.bind_visibility_to(data, 'show_grid')</code>

4. 其他常用方法

方法	作用	示例
<code>tooltip(text)</code>	为网格添加悬浮提示	<code>grid.tooltip('这是用户信息网格')</code>
<code>update()</code>	手动更新网格在客户端的显示	<code>grid.update()</code>
<code>mark(*markers)</code>	为网格添加标记（用于测试或查询）	<code>grid.mark('user-grid', 'info-section')</code>
<code>ancestors(include_self)</code>	迭代获取所有祖先元素	<code>for elem in grid.ancestors(): print(elem)</code>
<code>descendants(include_self)</code>	迭代获取所有子元素	<code>for child in grid.descendants(): print(child)</code>

六、常见使用场景与最佳实践

1. 数据展示表格（替代传统表格布局）

相比 `ui.table`，`ui.grid` 更灵活，适合非标准化数据展示：

```
from nicegui import ui

# 3 列布局：自适应标题列 + 2 列数据列
with ui.grid(columns='auto 1fr 1fr').classes('w-full gap-2 border p-2'):
    ui.label('产品').classes('font-bold')
    ui.label('价格').classes('font-bold')
    ui.label('库存').classes('font-bold')
    ui.label('手机')
    ui.label('¥5999')
    ui.label('100')
    ui.label('电脑')
    ui.label('¥9999')
    ui.label('50')
```

2. 复杂页面布局（组合多行多列 + 跨列）

实现“头部 + 侧边栏 + 内容区”布局：

```
from nicegui import ui

with ui.grid(rows='auto 1fr', columns='200px 1fr').classes('w-screen h-screen gap-0'):
    # 头部：跨 2 列
    ui.label('页面标题').classes('col-span-full bg-blue-500 text-white p-4 text-xl')
    # 侧边栏：200px 宽，占满剩余高度
    ui.label('侧边栏').classes('bg-gray-100 p-4')
    # 内容区：占满剩余空间
    ui.label('主要内容').classes('bg-gray-50 p-4')
```

3. 响应式布局（结合 Tailwind 响应式类）

利用 Tailwind 的响应式前缀（如 `md:`），实现不同屏幕尺寸下的布局切换：

```
from nicegui import ui

# 移动端 1 列，平板及以上 3 列
with ui.grid(columns='1fr md:3fr').classes('w-full gap-4'):
    ui.card('卡片 1').classes('p-4')
    ui.card('卡片 2').classes('p-4')
    ui.card('卡片 3').classes('p-4')
```

七、版本兼容性说明

- `html_id` 属性新增于 2.16.0 版本，低版本不支持。
- `toggle` 参数在 `default_classes()` 中新增于 2.7.0 版本。
- `strict` 参数在绑定方法（如 `bind_visibility`）中新增于 3.0.0 版本。
- 跨列语法 `col-[span_N]` 依赖 Tailwind 对任意值的支持，需确保 NiceGUI 版本兼容（建议 2.0+）。

总结

`ui.grid` 是 NiceGUI 中功能强大的布局组件，核心优势在于：

1. 行列配置灵活，支持数量、CSS 语法双重定义，适配各种布局需求；
2. 支持单元格跨列，轻松实现复杂页面结构；
3. 继承丰富的属性和方法，支持样式自定义、事件绑定、动态操作；
4. 与 Tailwind/Quasar 类无缝集成，响应式布局开发高效。

使用时需注意：行列的 CSS 语法需遵循 CSS Grid 规范，跨列时根据需求选择 Tailwind 内置类或自定义语法，同时关注版本兼容性以使用新增功能。